

Федеральное государственное бюджетное образовательное учреждение высшего образования «Херсонский государственный педагогический университет»
Институт информационных технологий
Направление подготовки: «09.04.04 Программная инженерия»
Специализация «Разработка программно-информационных систем»
Форма обучения: очная
Курс: 2, Магистратура, группа 12-25РПм
Херсонская область - 2026

Дисциплина:
«Управление конфигурацией и сопровождение программного обеспечения»

Выполнил:
Дмитрий Сергеевич Трифонов
обучающийся 2-го курса

Проверил:
Гуцол Д.В.

ПРАКТИЧЕСКАЯ РАБОТА 2

«Конфликт изменений, отмена ошибки и выпуск версии»

I. ЦЕЛЬ И ЗАДАЧИ

1.1. Цель

Целью данной практической работы является освоение методов управления изменениями и сопровождения программного проекта с использованием Git: создание независимых веток разработки, разрешение конфликта изменений, отмена незаписанной и сохранённой ошибки различными способами, подготовка версии проекта к выпуску и проверка возможности восстановления истории из переносимого Git bundle.

1.2. Задачи

- Создать локальный Git-репозиторий проекта с основной веткой main.
- Зафиксировать исходную версию проекта и создать тег v0.1.0.
- Создать две независимые ветки изменений с различными значениями параметра interval.
- Воспроизвести конфликт при объединении ветки feature-fast с веткой main.
- Разрешить конфликт с использованием согласованного значения interval=60.
- Отменить известную незаписанную ошибку с помощью git restore.
- Сохранить ошибочное изменение отдельным коммитом и отменить его с помощью git revert.
- Создать CHANGELOG.txt и подготовить версию v0.2.0.
- Создать аннотированный тег v0.2.0.
- Сформировать release.zip и переносимый Git bundle.
- Проверить корректность bundle и восстановить из него отдельную копию репозитория.
- Сохранить историю проекта в history.txt и подготовить итоговый отчёт.

II. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ

2.1. Управление конфигурацией с помощью Git

Git представляет собой распределённую систему контроля версий, позволяющую отслеживать изменения файлов, сохранять историю разработки и управлять различными линиями изменений. В данной работе Git используется локально, без подключения к удалённому серверу. Это позволяет обрабатывать операции управления конфигурацией непосредственно в рабочем каталоге проекта.

2.2. Ветки и независимые изменения

Ветка Git представляет отдельную линию развития проекта. В практической работе используются ветки main и feature-fast. Ветка feature-fast изменяет параметр interval с 60 до 30 секунд, а main независимо изменяет его с 60 до 120 секунд. Поскольку обе ветки изменяют одну и ту же строку исходного файла, при их объединении возникает конфликт.

2.3. Конфликт изменений

Конфликт возникает, когда Git не может автоматически определить, какое из независимых изменений должно попасть в итоговую версию. В данной работе main предлагает значение `interval=120`, а `feature-fast` — `interval=30`. Git сохраняет оба варианта в конфликтном файле и ожидает, что разработчик вручную определит согласованное содержимое. По условию задания итоговым значением является `interval=60`, поэтому оно записывается в файл перед завершением слияния.

2.4. Команды `git restore` и `git revert`

Команды `git restore` и `git revert` предназначены для разных ситуаций отмены изменений. `git restore` используется в работе для отказа от незаписанной рабочей правки: изменение `interval=0` не добавляется в индекс и не создаётся коммит, после чего файл восстанавливается до состояния индекса. `git revert` применяется уже после сохранения ошибочного изменения в истории: он создаёт новый коммит, отменяющий действие предыдущего ошибочного коммита. Таким образом, исходный ошибочный коммит остаётся в истории.

2.5. Версии, архивы и `Git bundle`

Теги Git позволяют отмечать определённые состояния проекта версиями. В работе исходное состояние отмечается тегом `v0.1.0`, а после исправления ошибки и подготовки `CHANGELOG` создаётся аннотированный тег `v0.2.0`. Команда `git archive` формирует архив файлов конкретной версии без истории Git. Команда `git bundle` сохраняет историю репозитория в переносимом файле, который можно использовать для последующего клонирования и восстановления проекта.

III. МЕТОДЫ И ПОДХОДЫ К ВЫПОЛНЕНИЮ РАБОТЫ

3.1. Подготовка рабочего окружения

Работа выполняется в Windows 10/11 с использованием Git Bash. Стартовый проект распаковывается отдельно от предыдущей практической работы. В каталоге `service-config` находятся `README.txt` и `config.txt`. Перед началом выполнения проверяется версия Git и текущее содержимое рабочего каталога.

3.2. Создание исходной версии

- Локальный репозиторий создаётся с основной веткой `main`.
- Настраиваются локальные имя и адрес электронной почты автора коммитов.
- `README.txt` и `config.txt` добавляются в индекс.
- Создаётся коммит `Add service configuration`.
- На исходное состояние проекта устанавливается тег `v0.1.0`.
- Проверяется, что `config.txt` содержит `interval=60` и рабочее дерево является чистым.

3.3. Формирование двух независимых изменений

- На основе main создаётся ветка feature-fast.
- В feature-fast значение interval изменяется с 60 на 30 секунд.
- Изменение фиксируется коммитом Set interval to 30 seconds.
- После возврата в main значение interval независимо изменяется с 60 на 120 секунд.
- Изменение main фиксируется коммитом Set interval to 120 seconds.
- Таким образом формируются две независимые линии изменения одного параметра.

3.4. Разрешение конфликта

- Из ветки main запускается слияние feature-fast.
- Git обнаруживает конфликт в config.txt, поскольку обе ветки изменили одну строку по-разному.
- В конфликтном файле присутствуют варианты interval=120 и interval=30.
- В качестве согласованного результата записывается interval=60.
- После проверки содержимого выполняется git diff --check.
- Изменённый файл добавляется в индекс и создаётся merge-коммит Merge interval settings: use 60 seconds.
- Проверяется, что рабочее дерево чистое, а merge-коммит имеет двух родителей.

3.5. Отмена незаписанной ошибки

После завершения слияния в рабочем файле намеренно создаётся ошибочная правка interval=0. Изменение проверяется через git diff, но не добавляется в индекс и не фиксируется коммитом. Команда git restore -- config.txt возвращает файл к состоянию индекса, после чего config.txt снова содержит interval=60. В результате новый коммит в истории не появляется.

3.6. Сохранение и отмена ошибочного коммита

Второй раз создаётся значение interval=0, после чего изменение добавляется в индекс и фиксируется коммитом Set invalid interval. Затем команда git revert --no-edit HEAD создаёт отдельный отменяющий коммит. В результате config.txt снова содержит interval=60, при этом исходный ошибочный коммит не удаляется из истории.

3.7. Подготовка версии v0.2.0

- Создаётся файл CHANGELOG.txt.
- В CHANGELOG.txt записывается номер версии v0.2.0.
- Добавляется описание разрешения конфликта с итоговым значением 60 секунд.
- Добавляется описание отмены ошибочного нулевого интервала.
- CHANGELOG.txt фиксируется коммитом Document v0.2.0 release.
- На состояние проекта после этого коммита устанавливается аннотированный тег v0.2.0.
- Проверяется наличие двух тегов: v0.1.0 и v0.2.0.

3.8. Формирование файлов передачи и проверка восстановления

- Создаётся каталог submission.
- Командой git archive формируется release.zip для версии v0.2.0.
- Командой git bundle create создаётся переносимый файл work.bundle со всей историей репозитория.
- Командой git bundle verify проверяется корректность bundle.
- Из work.bundle создаётся отдельная копия репозитория restore-check.
- В восстановленной копии проверяется interval=60, наличие тегов v0.1.0 и v0.2.0 и чистое рабочее дерево.
- История проекта сохраняется в history.txt.
- Итоговый report.txt помещается в каталог submission.

IV. ПРАКТИЧЕСКОЕ ВЫПОЛНЕНИЕ И АНАЛИТИЧЕСКОЕ ОСМЫСЛЕНИЕ

4.1. Исходная конфигурация и первая версия

- Исходный файл config.txt содержит значение interval=60.
- Создан локальный репозиторий с веткой main.
- Исходное состояние README.txt и config.txt зафиксировано коммитом Add service configuration.
- Создан тег v0.1.0, обозначающий исходную версию проекта.

4.2. Конфликт двух независимых изменений

- В ветке `feature-fast` параметр `interval` изменён на 30 секунд.
- В ветке `main` тот же параметр независимо изменён на 120 секунд.
- При выполнении `git merge feature-fast` Git не смог автоматически объединить изменения.
- Конфликт возник из-за изменения одной и той же строки в разных ветках.
- Для разрешения конфликта выбрано согласованное значение `interval=60`, предусмотренное условием задания.
- После выполнения `git add` и `commit` создан `merge-коммит` с двумя родителями.

4.3. Два способа отмены ошибки

- Незаписанная ошибка `interval=0` была отменена командой `git restore --config.txt`. Поскольку изменение не было сохранено коммитом, оно исчезло из рабочего дерева и не оставило отдельной записи в истории.
- Затем аналогичная ошибка `interval=0` была намеренно сохранена коммитом `Set invalid interval`.
- Для её отмены применена команда `git revert --no-edit HEAD`.
- После `revert` появился новый коммит, возвращающий `config.txt` к значению `interval=60`.
- Сравнение двух способов показывает различие между отменой локальной незаписанной правки и отменой уже сохранённого изменения.

4.4. Формирование и проверка выпуска

- После отмены ошибочного коммита создан `CHANGELOG.txt` с описанием изменений версии `v0.2.0`.
- `CHANGELOG.txt` сохранён отдельным коммитом `Document v0.2.0 release`.
- На итоговое состояние установлен аннотированный тег `v0.2.0`.
- Итоговая конфигурация содержит `interval=60`.
- Для точного маршрута задания итоговая история содержит 7 коммитов.
- Созданы `release.zip` и `work.bundle`.
- Команда `git bundle verify` подтвердила корректность переносимой истории.

4.5. Анализ release.zip и work.bundle

release.zip содержит файлы README.txt, config.txt и CHANGELOG.txt из отмеченной версии v0.2.0, но не содержит историю Git. work.bundle предназначен для переноса истории репозитория и позволяет выполнить клонирование проекта в новую папку. Проверка restore-check подтверждает, что из bundle восстанавливаются состояние interval=60, теги v0.1.0 и v0.2.0 и чистое рабочее дерево.

4.6. Аналитические вопросы

- Значение 60 секунд выбрано при разрешении конфликта потому, что оно является согласованным итогом по условию практической работы: проверка должна выполняться не реже одного раза в минуту, а более частые проверки пока не требуются.
- Чистое рабочее дерево само по себе не доказывает правильность настройки, поскольку git status показывает отсутствие незаписанных изменений, но не оценивает соответствие содержимого файла заданному условию.
- После revert ошибочный коммит остаётся в истории, а новый revert-коммит отменяет его эффект. Это позволяет сохранить полную последовательность действий.
- По work.bundle преподаватель может восстановить историю репозитория, проверить коммиты, ветки и теги. В release.zip содержится только набор файлов конкретной версии без истории Git.

V. ВЫВОДЫ

В результате выполнения практической работы были освоены методы управления изменениями и сопровождения программного проекта средствами Git. Была создана исходная версия проекта с тегом v0.1.0, сформированы две независимые ветки с различными изменениями параметра interval и воспроизведён конфликт при их объединении. Конфликт был разрешён выбором согласованного значения interval=60 с последующим созданием merge-коммита. На практике были сравнены два способа отмены изменений: git restore для незаписанной ошибки и git revert для уже сохранённого ошибочного коммита. После исправления была подготовлена версия v0.2.0 с CHANGELOG.txt и аннотированным тегом. Дополнительно были созданы release.zip, work.bundle и history.txt. Проверка bundle и клонирование восстановленной копии подтвердили возможность переноса и восстановления истории проекта. Практическая работа показала, что Git позволяет не только фиксировать изменения, но и безопасно управлять конфликтами, исправлять ошибки без потери истории, формировать версии программного продукта и обеспечивать восстановление состояния проекта.